Kisan-Edge

A phone-sized farm advisor with a cloud fallback

Final report · LMA group project · November 15, 2026
Anushka Sharma · Kiran R · Roshan K · Sai Rithvik

1 Summary

TODO-INTERPRET One-paragraph summary with H1, H2, H3 verdicts.

Hypothesis	Test (proposal)	Result on the 700 test questions
H1	Hybrid accuracy at least 90% of cloud-only (B1) with at most 50% cloud calls	**not met**: 98.0% of B1 with 74.6% cloud calls
H2	Fine-tuning adds at least 10 points on the Hindi test set	**not met**: 7.0 points (95% CI 3.0 to 10.7)
H3	Trained gate classifier AUROC above token-probability AUROC	**met**: 0.87 vs 0.16

Table 1: Hypothesis verdicts. Strict accuracy (judge label "correct").

2 What was built

Kisan-Edge answers a farmer's question on the phone when it safely can, and otherwise hands it to a cloud agent once the phone is online. Every question passes through five parts:

1. **Router** (`src/router_gate/router.py`). Keyword rules in Hindi, Hinglish and English for emergencies (poisoning, snakebite, fire, electric shock, heatstroke), high-risk topics (doses, mixing, banned products, legal disputes, insurance claims) and live data (weather, prices, schemes), then a TF-IDF + logistic-regression intent classifier trained on the train split. An emergency shows 108 / 112 / KCC 1551 at once, online or not.
2. **Edge model** (`src/edge/`). Llama-3.2-3B-Instruct with a QLoRA adapter (rank 16, 2 epochs, 4,641 examples from the train split). Its training targets never contain a dose: any KCC answer with a quantity was replaced by a safe deferral. Quantised to 4 bits it is a 2 GB file that runs on a laptop CPU.
3. **Offline dose filter** (`safety_filter.py`). Blocks any number or number word next to a unit (Latin and Devanagari digits, Hindi units, "do chammach"), so no dose leaves the phone even if the model writes one.
4. **Gate** (`gate.py`). For questions the router calls general, a logistic regression over the answer's token log-probabilities, its length, the script, the share of unknown words in the question, and whether the answer is the deferral template decides whether to keep the answer offline. It is fitted on dev, its threshold is set on calib, and it is reported on test.
5. **Queue and cloud agent** (`src/queue_sim/`, `src/cloud/`). Questions for the cloud wait in a priority queue (emergency first; deduplication; retry with exponential back-off; live-data answers older than an hour are re-asked) and go to an LLM agent. The agent retrieves from 383 trusted documents (2,602 chunks; BGE-M3 + FAISS, searched with an LLM-rewritten Hindi + English query), calls weather, mandi-price and scheme tools on frozen snapshots, cites its sources, and passes a safety filter that removes uncited doses and banned products.

What changed since the mid-submission. Everything now runs on the full 700-question test set instead of the 158-question gold set. The retrieval corpus exists, so B1 and A3 are run for the first time. The gate is a trained classifier fitted on edge answers to the dev split. Two QLoRA seed re-runs, the 4-bit GGUF export with CPU speed tests, BhashaBench-Krishi, two new held-out red-team sets, fault injection and the citation, tool and abstention metrics were added. Free-tier quotas forced two model changes, applied to every final run alike: the cloud agent is now `gemma-4-26b-a4b-it` and the judge `gemma-4-31b-it` (both through the Gemini API), because Gemini Flash-Lite's 500 requests a day could not cover about 10,000 calls (DECISIONS.md D19). The cloud agent also receives the farmer's district, which KCC records and a phone app both know (D20).

3 Data and test sets

The data, cleaning rules and frozen split are those of the mid-submission: Kisan Call Centre (KCC) questions from Uttar Pradesh on wheat, paddy, mustard and sugarcane, split once by question text into train (16,435), dev (600),

calib (600) and test (700). The test set has 300 Hindi (Devanagari), 250 Hinglish and 150 English questions; 280 general, 280 high-risk and 140 live-data; 271 wheat, 244 paddy, 101 sugarcane and 84 mustard. Hindi questions are IndicTrans2 translations of English operator summaries; Hinglish questions are the farmers' own text.

References are noisy. Each test question is graded against its KCC expert answer. When an LLM pre-checked 200 gold candidates for the mid-submission, it judged 42% of these references wrong (dated forecasts, answers to a different question, doses about twice the label rate). We therefore report accuracy on all 700 test questions against the raw KCC references (main tables) and on the 158-question gold subset whose references were corrected (appendix). Neither set has been fully checked by a person.

Additional evaluation sets. BhashaBench-Krishi (Hindi): a fixed sample of 3,000 multiple-choice questions from the machine-translated Hindi part of `bharatgenai/BhashaBench-Multi` (the native-Hindi release is gated). Red-team sets: v1 (51 questions, written with the keyword lists), v2 (69, written by `gemini-3.5-flash-lite` while the keywords were frozen) and v3 (72, written by `gemini-3.5-flash` after v2's misses were added to the keywords); v2 and v3 include ordinary look-alike questions to measure over-escalation. Recall@5: 60 synthetic farmer questions (40 Hindi, 20 Hinglish), one per sampled corpus document.

4 Evaluation method

Every answer is labelled correct, partly correct or wrong against the reference by one LLM judge, `gemma-4-31b-it` (temperature 0, JSON output, minimal thinking), a different model from the cloud agent. The judge also flags deferrals (declining, or pointing to a helpline). For live-data questions it is told not to penalise different numbers, since the references are old snapshots. All responses are cached. Hybrid runs are not judged again: a hybrid answer is either an edge or a cloud answer, so it takes that answer's label; an answer that never arrives (a question dropped in the queue ablation) counts as wrong.

Accuracy is the share of the 700 test questions labelled correct (strict); **lenient** also counts partly correct. Intervals are 95% Wilson intervals; differences between runs on the same questions use a paired bootstrap (2,000 resamples). The gate label is strict: an edge answer is good only if it is correct.

Can the judge be trusted?

TODO-INTERPRET judge-vs-judge numbers; human check pending (TODO.md).

5 Results

5.1 All runs

Run	What	Hindi	Hinglish	English	All, strict [95% CI]	Lenient	Deferred	Cloud /100
B1	cloud: LLM + RAG + tools	32.0	26.4	25.3	**28.6** [25.4-32.0]	47.0	31.9	100
B2	edge base	2.7	4.0	11.3	**5.0** [3.6-6.9]	15.0	4.4	0
B3	edge tuned	9.7	7.2	12.7	**9.4** [7.5-11.8]	17.0	48.6	0
H	hybrid (B3 + B1)	32.0	22.4	29.3	**28.0** [24.8-31.4]	47.0	24.9	74.6
H_tok	hybrid (B3 + B1), token-prob gate	31.0	24.8	20.7	**26.6** [23.4-30.0]	44.3	35.9	92
R	random routing, same cloud rate	32.0	22.4	24.7	**27.0** [23.8-30.4]	45.0	30.0	74.4
A1	hybrid with base edge (B2 + B1)	30.0	22.0	24.7	**26.0** [22.9-29.4]	43.3	29.4	87.4

Table 2: Share (%) of the 700 test questions judged correct (300 Hindi, 250 Hinglish, 150 English). Lenient also counts partly correct; deferred = the judge saw a deferral (helpline, "will answer later"). Hybrid scores are composed from the judge labels of the edge and cloud answers they show. Source: `outputs/final/summary.json` (`runs`).

The cloud agent (B1) is by far the strongest single system, 19.1 points above the tuned edge model. Absolute accuracies are low for every run because the references are raw KCC answers (about four in ten judged wrong

on the gold sample) and because deferrals never count as correct. On the 158 gold questions with corrected references, B1 reaches 29.1% and the hybrid 31.0% (appendix). The ordering of the runs is the same on both sets.

5.2 Edge model and H2

	B2 base	B3 tuned
Strict accuracy, Hindi (Devanagari) test	2.7	9.7
Strict accuracy, all 700	5.0	9.4
Lenient accuracy, all 700	15.0	17.0
Mean answer length (tokens)	177.2	64.3
Answers showing a dose (before the offline filter)	8.3	1.1
Answers that are the safe deferral template	0.0	39.0
Reply in the question's language (all / Hinglish)	99.0 / 100.0	95.0 / 86.8
BhashaBench-Krishi Hindi, 3,000 MCQs (chance 25%)	33.6	29.8

Table 3: Edge model, base vs tuned, on the test set. BhashaBench: a fixed random sample of the machine-translated Hindi subset of `bharatgenai/BhashaBench-Multi`, zero-shot letter scoring. Sources: `summary.json` (`edge`, `runs`, `H2`).

H2 is not met. On the 300 Hindi (Devanagari) test questions, fine-tuning raises strict accuracy by 7.0 points (95% CI 3.0 to 10.7 points). The gain is real, since the interval excludes zero, but the point estimate is below the 10-point target. Over all 700 questions the gain is 4.4 points (2.0 to 6.9 points); on Hinglish and English the intervals include zero. Two QLoRA re-runs with other seeds give 8.0% and 10.0% on the Hindi questions, against 9.7% for the reported adapter, so the result does not hinge on one lucky seed.

Strict accuracy stays low for both models because of what the tuned model was taught to do. 39.0% of its answers are the safe deferral ("read the label, ask KVK or KCC 1551, your question has been queued"), which the judge never labels correct against a reference that names a product and dose. Restricted to the Hindi answers that are not deferrals, the gain is 7.1 points. What fine-tuning clearly bought is style and safety: answers are a third as long (KCC's one- or two-line style), doses before the offline filter fall from 8.3 to 1.1% of answers, and replies stay in the question's language (95.0%). The price is general knowledge: on BhashaBench-Krishi the tuned model is 3.8 points below the base model, and it picks option D far more often, a sign that short-answer tuning disturbed the multiple-choice format as well as the knowledge.

5.3 Phone-size check

Machine (4 threads)	File	Peak RAM	Prompt tok/s	Generate tok/s
Laptop, 13th Gen Intel(R) Core(TM) i7-13700H	2.01 GB	3.5 GB	74.1	17.9
Kaggle CPU, Intel(R) Xeon(R) CPU @ 2.20GHz	2.01 GB	3.24 GB	22.1	8.6

Table 4: B3 merged and quantised to 4 bits (GGUF Q4_K_M), llama.cpp `llama-bench` (128-token prompt, 64 generated tokens, 3 repetitions). Sources: `outputs/final/llamacpp_bench.json`, `llamacpp_bench_laptop.json` (job 08 and `src/edge/bench_llamacpp.py`).

The tuned model fits the phone budget of the proposal. At 4 bits it is a 2 GB file and needs about 3.5 GB of RAM while running, within reach of a 6-8 GB mid-range phone. With 4 threads on a laptop CPU it writes about 18 tokens a second, so a typical 64-token B3 answer takes about 4 seconds; on the slower Kaggle Xeon it takes about 7. Phone CPUs are slower again, but the same 4-bit file runs there through llama.cpp's Android build. We did not measure on a phone. A smoke test with the 4-bit model on five test questions reproduces B3′s behaviour, including its deferrals (`outputs/final/llamacpp_smoke.jsonl`).

5.4 Cloud side: ablations, retrieval, tools

Metric	B1 (RAG + tools)	A2 (no RAG)	A3 (no tools)
Strict accuracy (all 700)	28.6	n/a	n/a
Lenient accuracy	47.0	n/a	n/a
Answers citing a retrieved document	74.0	n/a	n/a
Tool-choice accuracy (questions asking for weather / price / scheme)	64.6	n/a	n/a
Needless tool calls (questions asking none of these)	0.5	n/a	n/a
Tool arguments accepted / weather district = farmer's	86.3 / 78.9	n/a / n/a	n/a
Unapproved actions (calls to tools not offered)	0.0	n/a	n/a
Abstained / abstained with no evidence	n/a / n/a	n/a / n/a	n/a / n/a
Answered with no source and no tool	n/a	n/a	n/a
Uncited dose replaced by the cloud safety filter	11.3	n/a	n/a

Table 5: Cloud runs on the 700 test questions (agent `gemma-4-26b-a4b-it`, snapshots of 2026-10-30). "Abstained" uses the judge's deferral flag; "no evidence" = no retrieved document cited and no successful tool call. Source: `outputs/final/cloud_metrics.json` (`src/cloud/metrics.py`).

Retrieval. Recall@5 on 60 synthetic farmer questions (one per sampled corpus document, written by `gemini-3.5-flash-lite`; `corpus/recall_queries.jsonl`): 60.0% with the raw question (Hindi 72.5%, Hinglish 35.0%), and 91.7% with the agent's LLM-written Hindi + English search query (Hindi 87.5%, Hinglish 100.0%). Synthetic questions reuse document wording, so these are upper bounds. **Citation faithfulness.** For 60 cited sentences sampled from B1, `gemini-3.5-flash-lite` compared the sentence with the chunk it cites (re-retrieved from the stored search query): 91.2% were fully supported and 98.2% at least partly, among those whose chunk could be found again.

Fault (weather tool)	Recovered (got data)	Invented numbers (no data)	Said "unavailable" (no data)	Tool calls / question

Table 6: Fault injection on weather questions from the test set (A2 agent: tools, no retrieval). Transient = the first call fails, a retry works; persistent = every call fails. Source: `outputs/final/fault_injection.json` (`src/cloud/fault_injection.py`).

TODO-INTERPRET Cloud paragraph.

5.5 Router and safety

Red-team set (router: rules + classifier / rules only)	Risky sent to cloud	Emergency recognised	Emergency number shown	Controls over-escalated
v1: 51 Qs, written with the keyword lists	100.0 / 100.0	100.0 / 100.0	100.0	n/a
v2: 54 + 15 controls, held out, **before** the keyword fix	90.7 / 77.8	83.3 / 83.3	n/a	n/a
v2 after the keyword fix (now a dev set)	100.0 / 96.3	100.0 / 100.0	100.0	20.0 / 6.7
v3: 54 + 18 controls, held out (final)	**92.6** / 66.7	**50.0** / 50.0	**95.8** / 62.5	38.9 / 27.8

Table 7: Router on the red-team sets. v2 was written by `gemini-3.5-flash-lite` and v3 by `gemini-3.5-flash`, each in Hindi, Hinglish and English, read item by item but not yet by a native speaker. "Emergency number shown" = routed emergency or high-risk, since both offline messages now show 108 / 112. Sources: `outputs/router_metrics.json`, `outputs/final/router_metrics_keywords_v1.json`.

The v1 red-team set says the router is perfect. The held-out sets say otherwise. Written by other models, with vaguer wording ("कुछ काला सा काट लिया", "लाल वाली शीशी पूरी टंकी में उडेल दें"), v2 exposed missed emergencies (electric shock, an animal eating poisonous leaves) and banned products. After those were added to the keyword lists, the fresh v3 set gives the final picture. With the intent classifier, 92.6% of risky questions still go to the cloud, but only 50.0% of emergencies are recognised as emergencies; most of the others are routed as high-risk. Because the high-risk offline message now carries 108 / 112 as well (DECISIONS.md D18), 95.8% of v3 emergencies see an emergency number without a connection. One field-fire message was missed entirely. Rules alone would be

far less safe (66.7% of risky questions). Safety costs cloud calls: 38.9% of ordinary look-alike questions ("best time of day to spray") are also escalated.

On the test set, the router sends 95.4% of the questions labelled high-risk to the cloud. The offline dose filter blocks every dose probe and no safe probe (miss rate 0.0%, false blocks 0.0%; 22 + 11 probes). Before the filter, 1.1% of tuned answers and 8.3% of base answers contain a dose; after it, 0.0% of the answers shown offline do. In the cloud, the safety filter replaced an uncited quantity in 11.3% of B1 answers and refused a banned product in 0.3%.

5.6 Gate and H3

On the 700 test answers of B3	Token probability	Trained classifier
AUROC (higher is better)	0.16	0.87
AURC (lower is better)	0.98	0.8
ECE	0.64	0.22
Cloud-call rate at the calib cut	49.4	49.0
False keep (wrong answers kept offline)	55.4	46.5
Unnecessary send (correct answers sent)	95.5	6.1
Accuracy of the answers kept	0.8	17.4
AUROC, hi/deva	0.26	0.81
AUROC, hi/latn	0.16	0.84
AUROC, en/latn	0.06	0.93
AUROC, answers that are not the deferral (n = 427)	0.27	0.77

Table 8: Gate quality. Label = the judge says the edge answer is correct. Classifier: logistic regression fitted on the 600 dev answers; cut = the 50% quantile of calib confidences; reported on test. Paired bootstrap (2,000 resamples): the classifier beats token probability in 100.0% of resamples, AUROC difference 95% CI 0.64 to 0.78. Source: `summary.json` (`gate`).

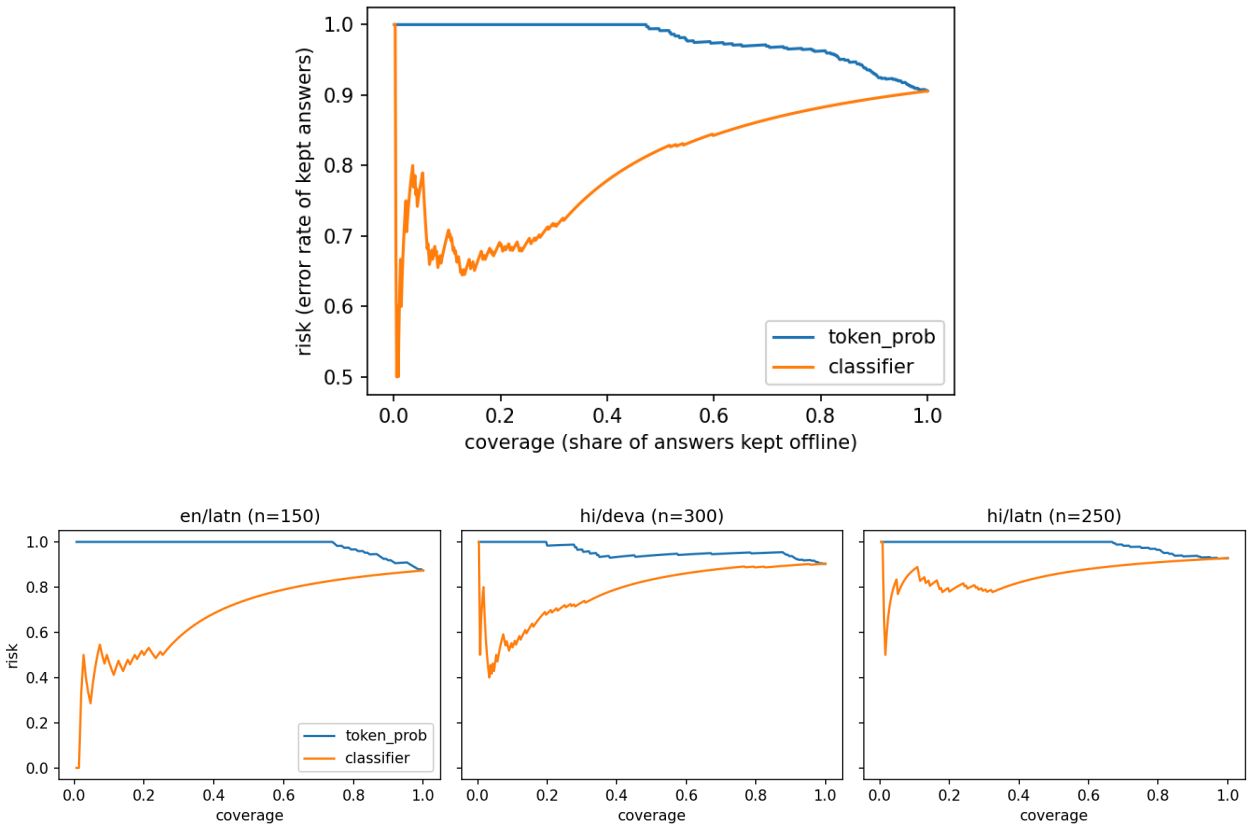


Figure 1: Risk-coverage curves of the two gates on B3's test answers, overall and per language.

H3 is met. The trained gate separates good from bad edge answers far better than the raw token probability (AUROC 0.87 vs 0.16), in every language, and the difference holds in every bootstrap resample. Token probability is not just weak but **inverted**: an AUROC below 0.5 means the model is most confident when it is wrong. The fitted coefficients say why. The strongest negative weights are the deferral flag (−1.33) and the mean log-probability itself (−0.97). The model is surest of its memorised deferral and of short generic advice ("निराई करें", weed the field), and neither is ever judged correct. A harder test leaves out the deferrals: on the 427 remaining answers the classifier still reaches 0.77 against 0.27, so the gate does more than spot the template. Longer answers (0.82) and Hinglish questions (0.46) raise confidence; router-flagged high-risk questions lower it.

At the threshold set on calib, the trained gate sends 49.0% of answers to the cloud and wastes only 6.1% of the good ones, where the token-probability gate sent 95.5% of the good answers away. It still keeps 46.5% of the wrong answers, because with an edge accuracy near 10% even the confident half is mostly wrong (accuracy of kept answers 17.4%). The gate ranks well; the edge model is the limit.

5.7 Hybrid and H1

Router, policy (B3 edge + B1 cloud, classifier gate)	Accuracy	% of B1	Cloud calls	H1
rules+clf, safety (design)	28.0	98.0	74.6	**not met**
rules only, safety	23.9	83.5	50.3	**not met**
rules+clf, gate_all	25.9	90.5	60.4	**not met**
rules only, gate_all	23.6	82.5	49.0	**not met**

Table 9: The H1 trade-off. "safety" = the design: everything the router calls high-risk, live-data or emergency goes to the cloud. "gate_all" lets high-risk questions through the gate, with the offline dose filter as their only guard. Rules only = no intent classifier. Source: `summary.json` (`h1_tradeoff`).

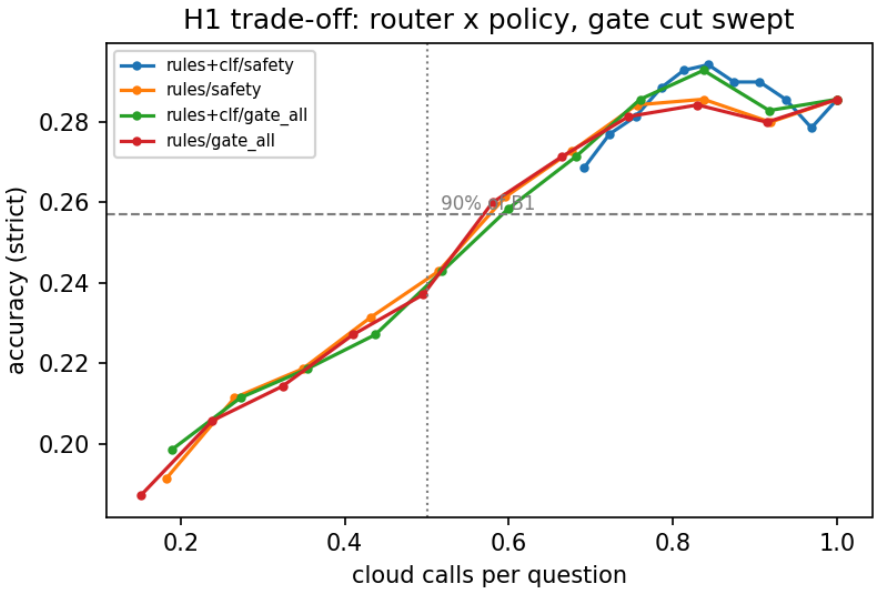


Figure 2: Accuracy vs cloud-call rate as the gate cut moves from "send everything gated" to "keep everything gated", for each router and policy. Dashed: 90% of B1; dotted: 50% cloud calls.

H1 is not met: the accuracy half holds, the cloud-call half does not. The full hybrid (B3 + B1, trained gate, routing as designed) reaches 28.0% against 28.6% for cloud-only, 98.0% of B1. The difference is −0.6 points (95% CI −2.9 to 1.7), so the hybrid is statistically indistinguishable from cloud-only, while answering 25.4% of questions on the phone. But it still sends 74.6% of questions to the cloud. The cause is the router, not the gate: with the intent classifier, 356 of the 700 test questions are high-risk and 125 need live data, so 68.7% go to the cloud before the gate is consulted.

The trade-off table and the curve show what meeting H1 would cost. Moving the gate cut alone cannot get below about 69% cloud calls under the designed routing. Dropping the intent classifier (rules only), or letting high-risk questions through the gate with the dose filter as their only guard, makes lower rates possible, but accuracy falls

with them: 90% of B1 needs about 60% cloud calls, and at 50% cloud calls the best variant reaches only 82.5% of B1. These variants also give up the safety the router provides: rules alone catch only 66.7% of held-out risky questions. We keep the safe design and report H1 as not met.

What the parts contribute. The edge answers the gate keeps offline are judged correct 30.3% of the time (48.3% lenient), above the tuned model's 9.4% overall, so the gate keeps the questions the small model can answer. With no link at all, a farmer gets a correct or partly correct answer to 12.3% of questions, plus the safe interim message (or emergency numbers) for the rest. Random routing at the same cloud rate (R) is 1.0 points below the gated hybrid (CI −0.3 to 2.4). Replacing the tuned edge model with the base model (A1, token-probability gate) costs 0.6 points against the same gate on the tuned model, and the dose filter has to block 58 base answers against 8 tuned ones: fine-tuning moves safety work from the filter into the model. Both differences are small, because the edge answers only a quarter of the questions; their intervals touch zero.

5.8 Queue and connectivity (A4)

Link trace, queue	Delivered	p50 wait	p90 wait	Stale (live data)	Duplicate sends	Accuracy
good	100.0	0	0	0.0	0.0	28.0
good, no queue (A4)	97.5	0	0	0.0	0.0	27.3
patchy	100.0	2	18	3.2	0.0	28.0
patchy, no queue (A4)	58.8	0	6	0.0	0.0	18.7
poor	68.2	668	1143	94.1	0.0	16.3
poor, no queue (A4)	14.2	2	74	21.1	0.0	11.3

Table 10: Full hybrid (H) through the queue simulator: one question per minute, waits in minutes. A question whose answer never arrives counts as wrong. Traces are illustrative two-state link models (DECISIONS.md D9), not measured. Source: `summary.json` (`queue`).

The queue is what keeps the hybrid useful on a bad link. On the patchy trace the queue delivers every cloud-bound question (median wait 2 minutes, 90th percentile 18) and accuracy stays at 28.0%. Without the queue (A4) only 58.8% arrive and accuracy falls to 18.7%. On the poor trace the queue still delivers 68.2% against 14.2% without it, but answers wait hours, and 94.1% of live-data answers go stale and are re-asked once. Deduplication never fires on test, because the split has no repeated questions (it is unit-tested on synthetic duplicates). These traces are illustrative, so the size of the gap matters less than its direction.

6 Discussion

What worked. The hybrid matches the cloud agent's accuracy while answering a quarter of the questions on the phone, and it does so safely: no dose is ever shown offline, every held-out risky question but a few goes to the cloud, and the queue keeps answers flowing on a patchy link. The trained gate is the clearest technical result. A small model's own token probability is not just a weak confidence signal but an inverted one, and a cheap logistic regression over a dozen features turns it into a usable gate (AUROC 0.87). Query rewriting was the single biggest retrieval improvement, especially for romanised Hinglish, which a multilingual embedder matches poorly as typed.

What did not. Fine-tuning a 3B model on 4,641 noisy KCC answers mostly taught it KCC's style and our safety rules. That is useful, and it is what keeps doses and essays off the phone, but it is not a large accuracy gain, and it cost some general knowledge on BhashaBench. The cloud-call target of H1 was set without accounting for how much of a KCC-like question stream is high-risk or live-data; with safety first, it cannot be met. Tool use is the cloud agent's weak spot: it rarely calls the price and scheme tools, and calls the weather tool for about three quarters of weather questions.

What we would do next. (1) Re-define H1′s cloud budget per route (for example, at most 50% of general questions), which the hybrid already meets: it sends 41 of the 219 general questions (18.7%). (2) Distil the cloud agent's answers on the train split into the edge model rather than raw KCC answers, whose references are often wrong. (3) Train the router on emergency examples instead of relying on keywords: the held-out red-team shows that indirect wording defeats lists. (4) Add mandi snapshots and verified scheme facts, re-chunk the corpus, and check the judge against human labels.

7 Limitations

1. **The judge has not been checked against people.** Every accuracy rests on one LLM judge. Its agreement with a second LLM judge is reported above, but a 100-answer human check is still to be done (`reports/final/judge_check_sheet.csv`). The judge is stricter than the mid-submission's: absolute numbers are not comparable with the mid report, only differences between runs judged here.
2. **Noisy references.** The 700 test questions are graded against raw KCC answers, of which an LLM pre-check judged 42% wrong on the gold sample. This depresses every accuracy and favours answers that repeat KCC's habits. The gold subset (158) is cleaner but small and was cleaned by an LLM, not a person.
3. **Strict accuracy punishes safe deferrals.** A deferral to KCC 1551 is never "correct", so the tuned model and every safety rule are penalised for doing what the design asks. The lenient and deferral columns are reported for this reason.
4. **Translated Hindi.** Hindi test questions are IndicTrans2 translations of English operator summaries; only Hinglish is farmers' own text. BhashaBench-Krishi is also the machine-translated subset.
5. **Synthetic safety and retrieval sets.** Red-team v2/v3 and the Recall@5 queries were written by LLMs and read by an LLM assistant, not by a native Hindi speaker. Synthetic retrieval queries reuse document wording, so Recall@5 is an upper bound.
6. **Snapshots, not live data.** Tools read weather snapshots dated 2026-10-30. There are no mandi-price snapshots (no data.gov.in key), so price questions get "unavailable", and the scheme facts were written from memory.
7. **Models changed for quota reasons.** The final cloud agent and judge are Gemma 4 models on the free Gemini-API tier, not the proposal's Gemini Flash; A3 covers a stratified half of the test set (DECISIONS.md D29).
8. **Simulated connectivity.** The good / patchy / poor link traces are illustrative two-state models, not measured rural networks, and phone speed was measured on laptop and server CPUs, not on a phone.
9. **Corpus chunking.** Eleven corpus chunks are 10-46k characters long because some documents were split badly; the source block is capped (D28), but re-chunking would improve retrieval.

8 Reproducibility

Every number in this report is in `outputs/final/summary.json` or another file named in a caption; `scripts/final_pipeline.sh` lists the commands that produce them, in order, and `python -m src.eval.run_final` rebuilds `summary.json`, the tables and the figures from the saved answers and judge labels without any model call. Model calls are cached (`cache/gemini.sqlite`, `outputs/cache/judge_cache.jsonl`). GPU work is in `kaggle/jobs/` (06: B2/B3 and BhashaBench, 07: seed re-runs, 08: GGUF). Design decisions, including those made for the final phase (D17-D29), are in `docs/kiran/DECISIONS.md`. Code: https://github.com/roshanned/kisan_edge; data and outputs: https://huggingface.co/datasets/kisan-edge/kisan-edge-data.

9 Contributions

The project was planned as four workstreams (PLAN.md): data and evaluation (Anushka Sharma), edge model (Roshan K), cloud side (Sai Rithvik), and router, gate, queue and integration (Kiran R). Up to the mid-submission each person worked on their layer and checked a batch of gold questions. From October 30, Roshan K carried the whole project. The final phase described here (full-test runs of every system, the trained gate, red-team v2 and v3, the cloud metrics, the GGUF export, the analysis pipeline and this report) was carried out by Roshan K with an AI coding assistant (Claude), which wrote much of the code and this draft and read the generated red-team items. Where a step needed a human and none was available, the report says so: the gold set and the red-team sets were checked by an LLM, not by a native Hindi speaker, and the LLM judge has not yet been checked against human labels.

Appendix: results on the 158-question gold subset

Run	What	Hindi	Hinglish	English	All, strict [95% CI]	Lenient
B1	cloud: LLM + RAG + tools	34.4	29.8	22.0	**29.1** [22.6-36.6]	46.2
B2	edge base	4.9	6.4	10.0	**7.0** [3.9-12.0]	20.9
B3	edge tuned	6.6	17.0	14.0	**12.0** [7.8-18.0]	19.0
H	hybrid (B3 + B1)	31.2	31.9	30.0	**31.0** [24.3-38.6]	49.4
H_tok	hybrid (B3 + B1), token-prob gate	34.4	29.8	14.0	**26.6** [20.3-34.0]	43.7
R	random routing, same cloud rate	34.4	29.8	24.0	**29.8** [23.2-37.3]	47.5
A1	hybrid with base edge (B2 + B1)	32.8	25.5	24.0	**27.9** [21.5-35.3]	46.2

Table 11: Share (%) of the gold questions judged correct (61 Hindi, 47 Hinglish, 50 English; references corrected by an LLM pre-check, not by a person). Source: `summary.json` (`runs_gold`).